Лабораторная работа № 5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP

Компьютер: AMD Ryzen 7 5700U (8 ядер / 16 потоков, 1,8 ГГц, L3 8 МБ), 15,3 ГБ ОЗУ, Windows; GCC (MSYS2), флаги **-O3 -march=native -fopenmp** (в 3.7 — разные). **Данные:** X ~ U(−1; 1), y = Xw* + 0,5 + шум, seed = 12345, тип double, у истинной модели ненулевые только 11 из 32 коэффициентов. **Параметры:** η = 0,1, λ = 10^{-3}, K = 100 итераций (в 3.6 K = 500), 16 потоков (в 3.5 — от 1 до 16). Время — медиана 3 запусков, измеряется только обучение (без генерации данных и вывода). Допуск для Δw: 10^{-8}. Разброс времени между запусками на ноутбуке достигает 20–30 %, поэтому разницу меньше этой величины не следует считать значимой.

3.1 Корректность (N = 1000, d = 8, 16 потоков)

Реализация	Итоговая функция потерь	Δw	Корректность подтверждена?
Последовательная	0,00681143	0	да
OpenMP	0,00681143	$5{,}2\cdot10^{-18}$	да
SIMD	0,00681143	$3{,}5\cdot10^{-18}$	да
OpenMP + SIMD	0,00681143	$5{,}2\cdot10^{-18}$	да

3.2 Сравнение реализаций (N = 10^5, d = 32, 16 потоков)

Вариант	Время, с	Ускорение S
Последовательная версия	0,429	1,00
OpenMP	0,189	2,27
SIMD	0,293	1,46
OpenMP + SIMD	0,181	2,37

3.3 Влияние числа объектов N (d = 32), время, с

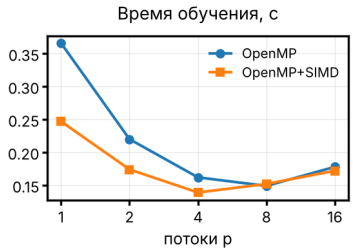
N	serial	OpenMP	SIMD	OpenMP + SIMD
10^4	0,041	0,067	0,021	0,067
10^5	0,399	0,174	0,251	0,168
10^6	4,147	1,254	2,974	1,206

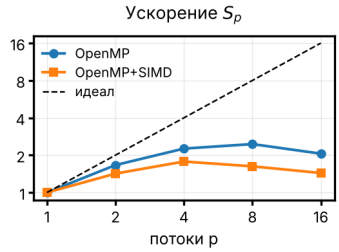
3.4 Влияние числа признаков d (N = 10^5), время, с

d	serial	OpenMP	SIMD	OpenMP + SIMD
8	0,118	0,096	0,134	0,099
32	0,452	0,185	0,363	0,179
128	2,549	0,558	1,563	0,542
256	5,132	1,000	3,058	1,007

3.5 Влияние числа потоков (N = 10^5, d = 32)

p	OpenMP, с	S_p	E_p	OpenMP+SIMD, с	S_p	E_p
1	0,366	1,00	1,00	0,247	1,00	1,00
2	0,220	1,66	0,83	0,174	1,42	0,71
4	0,162	2,26	0,56	0,139	1,78	0,44
8	0,149	2,46	0,31	0,152	1,63	0,20
16	0,178	2,06	0,13	0,172	1,44	0,09





3.6 Влияние λ (N = 10^5, d = 32, K = 500, ε = 10^{-6})

λ	Потери F(w, b)	Число $\lvert w_j\rvert < \varepsilon$	serial, с	OpenMP+SIMD, с
0	0,001672	0 из 32	1,81	0,90
10^{-4}	0,004422	14 из 32	2,19	0,95
10^{-3}	0,029155	21 из 32	1,81	0,87
10^{-2}	0,275019	21 из 32	1,92	0,88
10^{-1}	2,586554	21 из 32	1,87	0,88

3.7 Флаги компилятора (N = 10^5, d = 32, 16 потоков)

Флаги компиляции	serial, с	S	omp simd, с	S
-O0 -fopenmp	3,178	1,00	3,642	0,87
-O3 -fopenmp	0,632	5,03	0,457	6,95
-O3 -march=native -fopenmp	0,364	8,73	0,291	10,92

S — относительно первой строки (serial при -O0). Четвёртая строка задания (-O3 -march=native + omp simd) — это столбец «omp simd» последней строки. Каждая строка измерена отдельным запуском программы.

Выводы

1. **Самая быстрая — OpenMP + SIMD:** 2,37× при N = 10^5, d = 32; выигрыш растёт с размером задачи: 3,4× при N = 10^6 (4,147 → 1,206 с) и 5,1× при d = 256 (5,132 → 1,007 с). При малой задаче параллелизм невыгоден: при N = 10^4 OpenMP (0,067 с) медленнее serial (0,041 с), так как на каждой из K итераций создаётся параллельная область и сливаются 16 буферов градиента; там лучшей оказалась версия SIMD (0,021 с).

2. **OpenMP и OpenMP + SIMD** дают близкое ускорение (2,27× и 2,37×): при 16 потоках узким местом становятся память и накладные расходы, и вклад SIMD скрывается; на одном потоке SIMD помогает заметнее (0,366 → 0,247 с, в 1,5 раза). Ускорение меньше числа потоков, потому что матрица X (25,6 МБ) не помещается в кэш L3 (8 МБ) и целиком читается на каждой итерации, у процессора 8 физических ядер (16 потоков — это SMT), а параллельная область, calloc и critical повторяются на каждой итерации.

3. **Эффективность** падает: E = 1,00 → 0,83 → 0,56 → 0,31 → 0,13 (OpenMP) и 1,00 → 0,71 → 0,44 → 0,20 → 0,09 (OpenMP + SIMD). После 4–8 потоков время почти не улучшается, а при 16 растёт (0,149 → 0,178 с и 0,139 → 0,172 с): видны насыщение производительности и рост накладных расходов.

4. **Векторизация** (-fopt-info-vec, -O3 -march=native): GCC векторизовал все внутренние циклы по признакам — скалярное произведение и накопление градиента во всех четырёх вариантах (строки 51, 54, 74, 77, 100, 104, 125, 129), слияние буферов потоков (81, 133) — 32-байтными векторами (AVX2, 4 double), остатки — 16-байтными. Внешние циклы по объектам не векторизованы (диагностика -fopt-info-vec-missed: в них два последовательных внутренних цикла). Поскольку компилятор и так векторизует эти циклы, добавление omp simd даёт умеренный выигрыш: 0,364 → 0,291 с (1,25×). Выигрыш растёт с d: 0,88× при d = 8 (короткие циклы, затраты на остатки), 1,25× при d = 32, 1,63× при d = 128, 1,68× при d = 256. При -O0 omp simd не помогает (0,87×). Флаги дают основной вклад: -O3 ускоряет в 5×, -march=native — ещё в 1,7×.

5. **Регуляризация:** рост λ обнуляет коэффициенты: 0 из 32 при λ = 0, 14 при 10^{-4}, 21 при λ ≥ 10^{-3} — ровно столько нулей в истинной модели (21 из 32). Потери растут (0,0017 → 2,59) в основном из-за штрафа λ·Σ|w| (у истинной модели Σ|w| = 27,5), поэтому F при разных λ не сравнивается как качество модели. Время обучения от λ не зависит.

6. **Ограничивают производительность:** доступ к памяти (главный фактор: X читается целиком на каждой итерации), накладные расходы потоков и объединение частичных градиентов (заметны при малых N и d); для SIMD — короткие циклы по d. Сами вычисления (умножение-сложение), по-видимому, узким местом не являются.